

Tarea 1 – Manejo de ficheros binarios

Enunciado

1. ¿Recibe la clase `Scanner` sus datos de un stream?

Sí.

La clase `Scanner` recibe los datos desde un flujo de entrada (`InputStream`) o desde otras fuentes como archivos o cadenas de texto.

2. ¿Podría un fichero de disco ser un flujo de entrada?

Sí.

Un fichero puede actuar como flujo de entrada cuando un programa lee datos desde él usando clases como:
`FileInputStream`
`FileReader`

3. Hay muchos tipos de dispositivos de E/S. Algunos pueden ser origen, otros pueden ser destino y otros pueden ser tanto origen como destino. A menudo, el destino de un flujo es el origen de otro. En la tabla, decide qué papel puede jugar cada objeto.

Object	De origen, destino o ambos
Fichero de disco	Ambos
Programa en ejecución	Ambos
Monitor	Destino
Teclado	Origen
Conexión a internet	Ambos
Imagen escaneada	Origen

4. ¿Crees que es posible que un programa utilice varios flujos de entrada y varios flujos de salida?

Sí.

Un programa puede trabajar con múltiples flujos simultáneamente.

5. Enumera 3 tipos de ficheros que contienen flujos orientados a bytes.

- `.dat`
- `.jpg`
- `.mp3`

6. ¿La clase `Reader` es flujo de entrada o de salida?

`Reader` es un flujo de entrada orientado a caracteres.

7. ¿La clase `Writer` es un flujo de salida orientado a caracteres o orientado a bytes?

`Writer` es un flujo de salida orientado a caracteres.

8. ¿Se pueden escribir los 32 bits de una variable de tipo int directamente en un fichero binario sin traducirlos primero a caracteres?

Sí.

Con `DataOutputStream` se pueden guardar directamente los bits del entero sin convertirlos a caracteres.

9. El siguiente programa escribe 4 enteros en un fichero llamado "datos.dat":

```
try {
    DataOutputStream fos = new DataOutputStream(new
    FileOutputStream("datos.dat"));
    fos.writeInt(0);
    fos.writeInt(4);
    fos.writeInt(255);
    fos.writeInt(-7);
    fos.close();
} catch (FileNotFoundException e) { //todas pueden derivar de
IOException
    System.out.println(e.getMessage());
} catch (IOException e) {
    System.out.println(e.getMessage());
}
```

- a. Si el fichero ya existe, ¿Qué pasará con la información que tenía guardada? Razona tu respuesta.

El contenido anterior se sobrescribe porque `FileOutputStream` abre el archivo en modo escritura normal.

- b. ¿Qué tipo de flujo estamos utilizando?

Un flujo de salida binario orientado a bytes.

Clases usadas:

`FileOutputStream`

`DataOutputStream`

- c. El programa escribe cuatro valores int de 32 bits. ¿Qué tamaño tiene el fichero del disco?

Cada int ocupa 4 bytes.

Hay 4 enteros: $4 \times 4 = 16$ bytes

El fichero ocupa 16 bytes.

- d. ¿En qué directorio se ha creado el fichero?

En el directorio actual del proyecto o desde donde se ejecuta el programa.

- e. ¿Podemos usar un editor de texto, estilo Notepad++ para abrir y leer el fichero?

Se puede abrir, pero no leer correctamente porque es un fichero binario.

Aparecerán símbolos extraños.

- f. Crea el código para leer los enteros del fichero "datos.dat" y muestra la suma de los mismos.

```
import java.io.*;

public class LeerDatos {
    public static void main(String[] args) {

        int suma = 0;

        try {
            DataInputStream fis = new DataInputStream(
                new FileInputStream("datos.dat"));

            suma += fis.readInt();
            suma += fis.readInt();
            suma += fis.readInt();
            suma += fis.readInt();

            fis.close();

            System.out.println("La suma es: " + suma);

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

10. ¿Es correcta la siguiente línea de código?

```
DataOutputStream output = new DataOutputStream("file.dat" );
```

No.

DataOutputStream necesita un *OutputStream*.

Correcto:

```
DataOutputStream output =
    new DataOutputStream(new FileOutputStream("file.dat"));
```

11. Accede a la documentación oficial de la clase *DataOutputStream*, ¿cuántos bytes escribirán las siguientes sentencias?

```
output.writeBoolean(true);
output.writeBoolean(false);
```

Cada boolean ocupa 1 byte.

15. Crea un programa que lea por parámetros de entrada un fichero de entrada y un fichero de salida y copie el contenido del fichero de entrada en el de salida.

Ejemplo: *java sourceFile destinationFile*

```
import java.io.*;

public class CopiarFichero {

    public static void main(String[] args) {

        if (args.length != 2) {
            System.out.println("Uso: java CopiarFichero origen destino");
            return;
        }

        String origen = args[0];
        String destino = args[1];

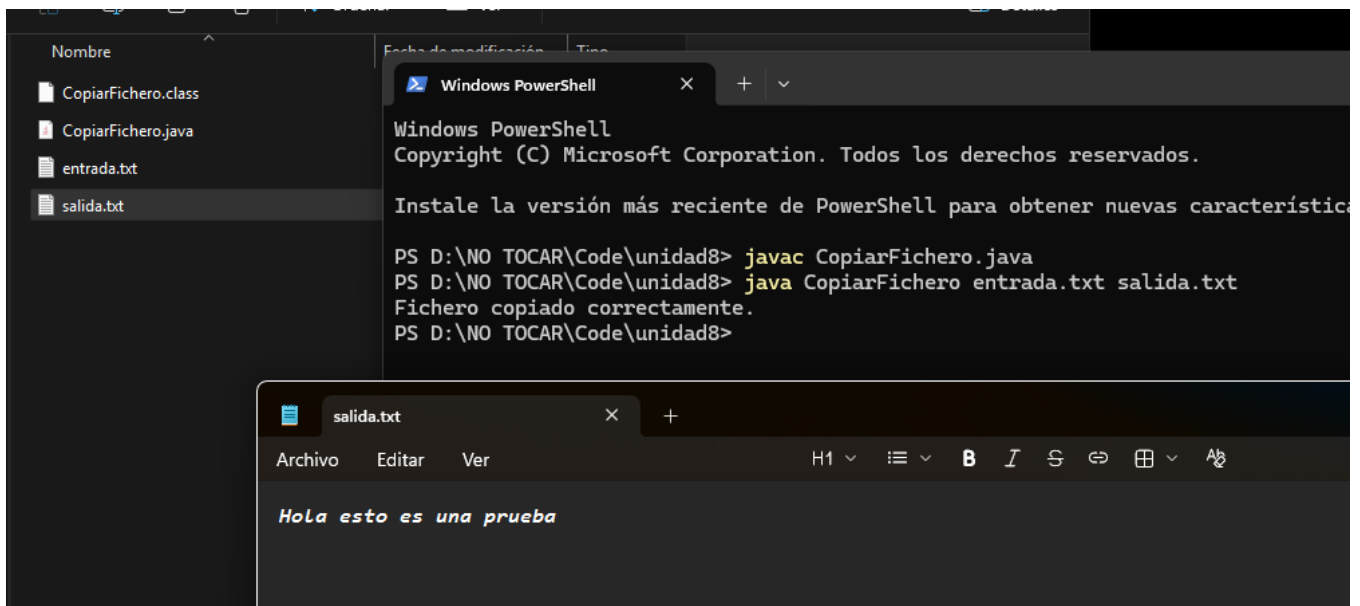
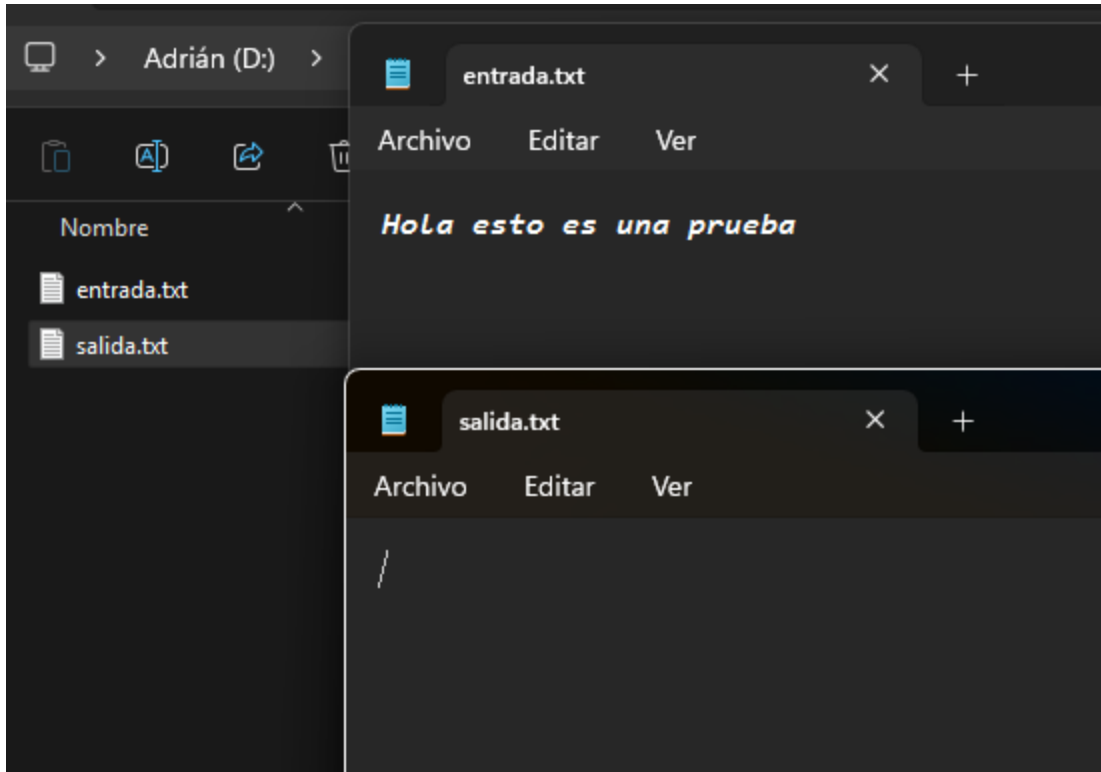
        try (
            FileInputStream fis = new FileInputStream(origen);
            FileOutputStream fos = new FileOutputStream(destino)
        ) {

            int dato;

            while ((dato = fis.read()) != -1) {
                fos.write(dato);
            }

            System.out.println("Fichero copiado correctamente.");

        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```



Entrega

Sube un PDF con las preguntas respondidas, copia el código en el PDF y añade capturas de pantalla del resultado del código ejecutado.